Oracle C2M Hypercare & Production Support Runbook (Go-Live Readiness)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M Hypercare & Production Support Runbook 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Oracle C2M Hypercare & Production Support Runbook 5

What hypercare is . . . 5

Why this matters . . . 5

The hypercare operating model . . . 6

The production-support structure . . . 6

Named objects this solution defines . . . 7

What this is not . . . 7

The deployment model . . . 7

2. Requirements Guide — Oracle C2M Hypercare & Production Support Runbook 8

2.1 A reachable C2M environment and product user . . . 8

2.2 A go-live readiness checklist and its owners . . . 8

2.3 Monitoring and health-check access . . . 8

2.4 On-call rota and contacts defined . . . 9

2.5 Escalation matrix and severity model . . . 9

2.6 Incident and known-issue handling process . . . 9

2.7 Lower-environment access for day-2 build support . . . 9

2.8 Cutover plan, backups, and rollback . . . 10

2.9 A non-production dress rehearsal and change control . . . 10

Checklist . . . 10

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 11

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 11

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 11

3. Environment topology (Dev / Test / Prod) & access . . . 11

4. Administrative access & tooling (WLST, shell, SQL) . . . 12

5. Change control, backup & a non-production pilot . . . 12

6. Collaborating teams & support model . . . 12

3. Runbook — Oracle C2M Hypercare & Production Support 13

Step 1 — Build the go-live readiness checklist (`C2M Runbook - Readiness - Go-Live Checklist`) . 13

Step 2 — Author the cutover plan (`C2M Runbook - Cutover - Plan`) . . . 14

Step 3 — Define the severity model (`C2M Runbook - Severity - Model`) . . . 14

Step 4 — Build the on-call rota and escalation matrix . . . 14
Step 5 — Adopt the incident procedure (`C2M Runbook - Incident - Procedure`) . . . 15
Step 6 — Handle known issues (`C2M Runbook - KnownIssue - Register`) . . . 15
Step 7 — Set up the day-2 build lane (`C2M Runbook - Day2 - Build Lane`) . . . 15
Step 8 — Configure the daily health/monitoring routine (`C2M Runbook - Health - Daily Routine`) . 15
Step 9 — Pilot the whole model on a non-production dress rehearsal . . . 16
Step 10 — Expand into the live cutover and hypercare . . . 16
5. Testing Guide — Oracle C2M Hypercare & Production Support Runbook 17
Test 1 — Readiness is actually gated . . . 17
Test 2 — Cutover is reversible . . . 17
Test 3 — On-call and escalation reach the right owner . . . 17
Test 4 — An incident is handled and recorded . . . 17
Test 5 — A known issue is recognized, not re-diagnosed . . . 18
Test 6 — Day-2 builds stay in the lower environments . . . 18
Test 7 — The daily health routine produces a clean snapshot . . . 18
Test 8 — Health is stable across the hypercare window . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — Oracle C2M Hypercare & Production Support Runbook 19
A. Readiness checklist stays red . . . 19
B. Cutover go/no-go fails . . . 19
C. An incident has no clear owner . . . 19
D. The same problem keeps being re-diagnosed . . . 19
E. Day-2 change destabilized production . . . 20
F. Daily health shows amber/red . . . 20
G. JMX batch connector cannot connect . . . 20
H. Health will not stabilize / hypercare will not exit . . . 21
I. Diagnostics quick reference . . . 21
J. When to escalate . . . 21
7. Best Practices — Oracle C2M Hypercare & Production Support Runbook 21
8. Deliverables Summary — Oracle C2M Hypercare & Production Support Runbook 22
Documentation . . . 22
Named objects . . . 23
Automation scripts (WLST/Jython, shell & SQL — not PowerShell) . . . 23
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Oracle C2M Hypercare & Production Support Runbook

The riskiest weeks of any Oracle Utilities Customer to Meter (C2M) program are **go-live and hypercare** — the readiness checks before cutover, the cutover itself, and the elevated-support window that follows while the team runs live production for the first time. Done without a runbook, these weeks are where good programs stumble: cutover begins on hope, the first incident is triaged by whoever is watching, the same problem is diagnosed three times, day-2 work either freezes or destabilizes production, and the first sign of trouble is a business user's phone call. This solution is the **hypercare and production-support runbook** that removes that risk: a go-live readiness gate, the on-call and escalation model, incident and known-issue handling, day-2 build support in the lower environments, and the daily health/monitoring routine — so the team runs production confidently from cutover through stabilization.

This is a complete, do-it-today package for an experienced C2M production-support lead. Every structure is named once — the readiness checklist, the cutover plan, the on-call rota, the escalation matrix, the severity model, the incident procedure, the known-issue register, the day-2 build lane, the daily health routine — and reused from cutover to steady-state, so you configure it once and run it every day of hypercare.

What you get

• A plain-language Solution Overview of the hypercare operating model (readiness → cutover → hypercare → steady-state) and the production-support structure, with an explicit gate at each phase.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — a reachable environment, the readiness checklist with owners, monitoring/health-check access (WLST, the JMX batch connector, the schema, the logs), the on-call rota, the escalation matrix and severity model, the incident procedure and known-issue register, lower-environment access for day-2, and the cutover plan with backups and rollback.

• The Runbook itself: the go-live readiness checklist, the on-call/escalation model, incident and known-issue handling, day-2 build support in the lower environments, and the daily health/monitoring routine — each with a pass/fail gate, pilot-first on a non-production dress rehearsal.

• Five automation scripts (shell, WLST/Jython, and SQL — not PowerShell) for readiness/health discovery, go-live readiness validation, a health and incident report, safe remediation (dry-run by default), and a scope/target helper (rota and day-2 lane) — authentic to C2M and WebLogic administration.

• Testing and Troubleshooting guides with explicit pass/fail gates (readiness gated, incidents handled, health stable), plus Best Practices for a smooth stabilization.

What this delivers

A C2M go-live and hypercare the team can actually run — readiness proven, on-call structured, incidents handled, health stable — from cutover through stabilization.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — rehearsed on a non-production dress rehearsal before the live cutover, with a documented rollback for every production change.

Outcome

Reliable C2M stabilization: a shaky, ad-hoc go-live — no readiness gate, ad-hoc on-call, unhandled incidents, chaos at cutover — replaced by a runbook-driven cutover and hypercare with a proven readiness gate, a structured on-call and escalation model, handled and recorded incidents, controlled day-2 builds, and a green daily health snapshot, exiting to steady-state on evidence rather than the calendar.

1. Solution Overview — Oracle C2M Hypercare & Production Support Runbook

What hypercare is

The riskiest weeks of any Oracle Utilities Customer to Meter (C2M) program are the ones around **go-live**: the days spent proving the environment is ready, the **cutover** itself, and the **hypercare** period that follows — the elevated-support window where the team runs live production for the first time, watches it closely, and stabilizes it before handing it to steady-state operations. Hypercare is not a mood ("everyone pay attention"); it is an **operating model**: a readiness gate that says go/no-go, an on-call and escalation structure so every issue has an owner and a path, a disciplined way to handle incidents and known issues, a way to keep building in the lower environments while production runs, and a **daily health routine** so the state of the system is a measured fact each morning, not a rumor. This solution is that operating model as a **runbook** — named checklists and rotas you define once and reuse from cutover through stabilization.

> ***Version boundary.*** *Command names, script names, monitoring interfaces, and object terminology below are from the Oracle Utilities Application Framework (OUAF) Server Administration Guide and the C2M product documentation. Version-specific details (C2M 25.4 / minimum v2.9, WebLogic 14c, the JMX batch connector,* `threadpoolworker/starttpw`*) are* ***cited or flagged "verify for your version"*** *— never invented. Go-live checklists, patch bundles, and known-issue notes are frequently gated on My Oracle Support (MOS); those are* ***pointed to, not reproduced****. See* `global-requirements-oracle.md`*.*

Why this matters

Go-live without a runbook is where good programs stumble. Without a **readiness gate**, cutover begins on hope — an unpatched node, an unconfigured batch daemon, or a missing integration endpoint surfaces at the worst possible time. Without a **defined on-call and escalation model**, the first production incident is triaged by whoever happens to be watching, and the bridge call spends its first hour deciding who owns what. Without disciplined **incident and known-issue handling**, the same problem is diagnosed three times by three people and the fixes are not written down. Without **day-2 build support in the lower environments**, the team either freezes all further work (and falls behind) or builds in production (and destabilizes it). And without a **daily health routine**, the first sign of trouble is a business user's phone call. A real hypercare model fixes the cause: it gates readiness, structures on-call, handles incidents once and records them, keeps day-2 work safely in lower environments, and makes health a measured morning fact

— so the team runs production confidently from cutover.

The hypercare operating model

Four phases, each with an explicit gate before the next begins:

1. **Readiness** — before cutover, prove the environment, the data, the integrations, the batch

estate, the monitoring, and the support structure are all go. The **Go-Live Readiness Checklist** (`C2M Runbook - Readiness - Go-Live Checklist`) is the gate: no red items, or a signed-off exception, before cutover starts.

1. **Cutover** — execute the cutover plan (final data load/sync, config promotion, integration

enablement, batch daemon start) against a timeboxed plan with a **go/no-go** decision point and a documented **rollback**. Cutover ends when the smoke checks pass and production is accepting work.

1. **Hypercare** — the elevated-support window. The **on-call rota** (`C2M Runbook - OnCall - Rota`)

is staffed, the **escalation matrix** (`C2M Runbook - Escalation - Matrix`) is live, incidents run through the **incident procedure**, the **known-issue register** (`C2M Runbook - KnownIssue - Register`) is kept current, day-2 builds continue in lower environments, and the **daily health routine** (`C2M Runbook - Health - Daily Routine`) runs every morning.

1. **Steady-state** — when the exit gate is met (health stable for the agreed number of consecutive

days, incident rate below threshold, no open Sev-1/Sev-2, day-2 backlog controlled), hypercare **exits** to normal operations and the runbook becomes the standing operations runbook.

The production-support structure

Hypercare runs on named, reusable structures rather than ad-hoc heroics:

• **On-call rota** — `C2M Runbook - OnCall - Rota`: the primary/secondary schedule for each support

window, by layer (application, middleware/WebLogic, database, integration, infrastructure), with a handover step between shifts.

• **Escalation matrix** — `C2M Runbook - Escalation - Matrix`: for each severity, who is engaged,

in what order, within what time, up to the incident commander and — for product defects — the Oracle Service Request path on MOS.

• **Severity model** — `C2M Runbook - Severity - Model`: the definitions (Sev-1 … Sev-4) that decide

response time, who is paged, and communications cadence, so severity is a rule, not an argument.

• **Incident procedure** — `C2M Runbook - Incident - Procedure`: detect → triage → assign owner →

diagnose → mitigate/resolve → communicate → record, with the bridge/comms pattern for a major incident.

- **Known-issue register** — `C2M Runbook - KnownIssue - Register`: the living list of known defects

and their workarounds (many tracked to a MOS note or patch), so a recurring symptom is recognized and worked around in minutes, not re-diagnosed.

- **Day-2 build lane** — `C2M Runbook - Day2 - Build Lane`: how further configuration/development

continues safely in the lower environments (Dev/Test/UAT) during hypercare and promotes to production through change control, never bypassing it.

- **Daily health routine** — `C2M Runbook - Health - Daily Routine`: the fixed morning set of

read-only checks (WebLogic servers, batch completion and backlog, integration queues, database and disk, error-log deltas) that produces a green/amber/red health snapshot every day.

Named objects this solution defines

So the runbook is repeatable, this solution names its artifacts once and reuses them:

- `C2M Runbook - Readiness - Go-Live Checklist` — the go/no-go readiness gate.
- `C2M Runbook - Cutover - Plan` — the timeboxed cutover sequence with its go/no-go and rollback.
- `C2M Runbook - OnCall - Rota` — the primary/secondary on-call schedule by layer.
- `C2M Runbook - Escalation - Matrix` — who is engaged, in what order, by severity.
- `C2M Runbook - Severity - Model` — the Sev-1…Sev-4 definitions and response targets.
- `C2M Runbook - Incident - Procedure` — the standard incident lifecycle.
- `C2M Runbook - KnownIssue - Register` — known defects and their workarounds.
- `C2M Runbook - Day2 - Build Lane` — day-2 build support in lower environments.
- `C2M Runbook - Health - Daily Routine` — the daily read-only health snapshot.

What this is not

This solution provides the **operating model and runbook** for C2M go-live and hypercare; it does not replace the environment build, the batch scheduling design, the monitoring platform, or the integration configuration — those are their own disciplines and are referenced, not restated. Where the included scripts touch live systems they **read**: the discovery, validation, and report scripts are read-only; remediation runs **dry-run by default** and only prints or applies safe operational alignments (never business logic) under change control. Every production change during cutover and day-2 goes through change control and is proven on a lower environment first.

The deployment model

> *Assess readiness against the checklist, configure the support structure (rotas, escalation, severity, incident procedure, known-issue register, day-2 lane, health routine), pilot the whole model on a non-production dress rehearsal, validate the readiness and health gates, then expand into the live*

cutover and hypercare. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.

2. Requirements Guide — Oracle C2M Hypercare & Production Support Runbook

Verify every requirement before you gate cutover and stand up hypercare. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (licensing, certified stack, environments, WLST/shell/SQL tooling, change control, collaborating teams and support model) are in `global-requirements-oracle.md`.

2.1 A reachable C2M environment and product user

Description: The production C2M environment (and its OUAF product home) reachable, with the OS product user that owns it — the account under which `threadpoolworker`/`starttpw` and the batch daemon run — and the environment sourced via `splenviron.sh -e your-env`. Why: Every readiness, health, and validation script needs the environment variables `splenviron.sh` sets; without them the utilities cannot locate the WebLogic domain, the class path, or the configuration. Hypercare's daily routine runs as this user. Verify: As the product user, run `splenviron.sh -e your-env` then run `OC2MHYPER-Discovery.sh` — it confirms the environment is sourced and the OUAF/WebLogic utilities are on the path. Implement-if-missing: Obtain the product-user credentials and the environment name; confirm the OUAF install is complete (`initialSetup`/`configureEnv` run) before proceeding.

2.2 A go-live readiness checklist and its owners

Description: A defined **Go-Live Readiness Checklist** (`C2M Runbook - Readiness - Go-Live Checklist`) covering environment, data/migration sign-off, integrations, batch estate, security/SSO, monitoring, backup/rollback, and support readiness — each item with an accountable owner and a go/no-go status. Why: Cutover must begin from a gate, not a hunch; the checklist is what turns "are we ready?" into a signed go/no-go with named owners for every open item. Verify: The checklist exists with an owner per item; run `OC2MHYPER-Validation.sh`, which checks the technical readiness signals (WebLogic servers, batch daemon, JMX, integration reachability) that back the checklist. Implement-if-missing: Build the checklist from this runbook (Implementation Step 1) and the program's cutover plan; assign an owner to each line and to any pre-approved exception.

2.3 Monitoring and health-check access

Description: Read access to the signals the **daily health routine** depends on: the WebLogic Administration Console / WLST for server state, the **JMX batch connector** (`service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector` — verify for your version) for live batch threads, the C2M schema for batch run status and integration queues, and the OUAF/WebLogic log locations, plus whatever monitoring platform (e.g. Dynatrace) the environment uses. Why: Hypercare closes the loop by measuring health daily; without read access to servers, batch, the schema, and logs there is no `C2M Runbook - Health - Daily Routine` and problems surface from the business instead of from operations. Verify: You can reach WLST, query batch run status, and read the logs; run `OC2MHYPER-Report.sql` (read-only) and confirm it returns recent batch and integration status.

Implement-if-missing: Grant the read-only reporting access (or use the product user); confirm WLST, JMX, SQL, and log-path reachability per the global requirements.

2.4 On-call rota and contacts defined

Description: A staffed **on-call rota** (`C2M Runbook - OnCall - Rota`) — primary and secondary per support window, by layer (application, middleware/WebLogic, database, integration, infrastructure) — with current contact details and a shift-handover step. Why: An incident with no named owner burns its first hour deciding who owns it; the rota makes ownership immediate and continuous across shifts through hypercare. Verify: The rota is published for the hypercare window with contacts for every layer; run `OC2MHYPER-ScopeHelper.sh --scope rota` to render the current-window contact list from the rota file. Implement-if-missing: Build the rota with the collaborating teams (global requirement 6); publish it where the whole team can reach it and confirm every layer has a primary and a secondary.

2.5 Escalation matrix and severity model

Description: A defined **severity model** (`C2M Runbook - Severity - Model`, Sev-1…Sev-4 with response targets) and an **escalation matrix** (`C2M Runbook - Escalation - Matrix`) mapping each severity to who is engaged, in what order, within what time — up to the incident commander and the Oracle Service Request path on My Oracle Support for product defects. Why: Severity decides response time, who is paged, and comms cadence; without a rule it is argued live during the incident. The matrix ensures the right people are engaged fast and the MOS path is known. Verify: Severity definitions and the escalation matrix exist and are agreed; a tabletop walk of one Sev-1 and one Sev-3 reaches the right owners within target. `OC2MHYPER-Validation.sh` confirms the runbook files are present and non-empty. Implement-if-missing: Adopt the severity model and matrix from this runbook (Implementation Steps 3-4) and confirm the MOS account and SR process for defect escalation.

2.6 Incident and known-issue handling process

Description: A standard **incident procedure** (`C2M Runbook - Incident - Procedure`: detect → triage → assign → diagnose → mitigate/resolve → communicate → record) and a **known-issue register** (`C2M Runbook - KnownIssue - Register`) that captures recurring defects and their workarounds, many tracked to a MOS note or patch. Why: Without a procedure the same incident is diagnosed repeatedly and fixes are not recorded; without a register a known symptom is re-diagnosed from scratch each time instead of worked around in minutes. Verify: The procedure and an initialized register exist; run `OC2MHYPER-Report.sh` to render the current open-incident / known-issue summary from the register file (read-only). Implement-if-missing: Adopt the procedure and seed the register from testing/UAT defects and the version's known-issue MOS notes before go-live.

2.7 Lower-environment access for day-2 build support

Description: Access to the **lower environments** (Dev/Test/UAT) so day-2 configuration and development continue during hypercare without touching production — the `C2M Runbook - Day2 - Build Lane` — with promotion to production only through change control (per the global requirements and any existing code-promotion process). Why: Hypercare does not stop the roadmap; day-2 work must keep flowing, but it must flow through lower environments and change control, never directly into a stabilizing production system. Verify: You can reach the lower environments and a promotion/change-control path to production exists; `OC2MHYPER-ScopeHelper.sh --scope day2` lists the environments and controls in the day-2

lane. Implement-if-missing: Obtain lower-environment access and confirm the change-control/promotion path before hypercare begins.

2.8 Cutover plan, backups, and rollback

Description: A timeboxed **cutover plan** (`C2M Runbook - Cutover - Plan`) with a go/no-go decision point, verified **backups** (WebLogic domain config, OUAF product/config, database) taken before cutover, and a documented **rollback** to the pre-cutover state. Why: Cutover is the single highest-impact change of the program; it must be sequenced, timeboxed, gated with a go/no-go, and reversible. A rollback that is not written down is not a rollback. Verify: The cutover plan, a confirmed backup/restore path, and a documented rollback all exist and have been walked in a dress rehearsal on non-production. Implement-if-missing: Author the cutover plan, take and test the backups, and document the rollback — and rehearse the whole sequence on a lower environment before the live cutover.

2.9 A non-production dress rehearsal and change control

Description: A non-production environment to **rehearse** the cutover and the daily health routine, and prove the support structure end to end before go-live, plus change control and backups per the global requirements. Why: The model is Assess → Configure → Pilot → Validate → Expand; the pilot is a dress rehearsal that finds the gaps while they are cheap, not during the live cutover. Verify: A non-production environment is available and the cutover, health routine, and an incident tabletop have been rehearsed on it with a documented rollback. Implement-if-missing: Stand up or obtain the non-production environment and schedule the dress rehearsal before the go-live window.

Checklist

- C2M environment reachable, product user, `splenviron.sh -e your-env` sources cleanly — `OC2MHYPER-Discovery.sh`
- Go-live readiness checklist with owners — `C2M Runbook - Readiness - Go-Live Checklist`, `OC2MHYPER-Validation.sh`
- Monitoring/health-check access (WLST, JMX batch connector, schema, logs) — `OC2MHYPER-Report.sql`
- On-call rota with contacts by layer — `C2M Runbook - OnCall - Rota`, `OC2MHYPER-ScopeHelper.sh --scope rota`
- Escalation matrix + severity model — `C2M Runbook - Escalation - Matrix`, `C2M Runbook - Severity - Model`
- Incident procedure + known-issue register — `C2M Runbook - Incident - Procedure`, `OC2MHYPER-Report.sh`
- Lower-environment access for day-2 builds — `C2M Runbook - Day2 - Build Lane`, `OC2MHYPER-ScopeHelper.sh --scope day2`
- Cutover plan, backups, documented rollback — `C2M Runbook - Cutover - Plan`
- Non-production dress rehearsal + change control

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target

environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Runbook — Oracle C2M Hypercare & Production Support

This is the runbook: the go-live readiness checklist, the on-call/escalation model, incident and known-issue handling, day-2 build support in the lower environments, and the daily health/monitoring routine — each with an explicit gate. Follow it in order from readiness through cutover into hypercare and out to steady-state. Object names are fixed — create them with the exact names shown and reuse them everywhere. Pilot the whole model on a non-production dress rehearsal before the live cutover.

> *Model: assess readiness, configure the support structure, rehearse (pilot) on non-production, validate the readiness and health gates, then expand into the live cutover and hypercare. Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.*
>
> *All commands assume the environment is sourced first:* `splenviron.sh -e your-env` *(run as the product user). Utility and interface names (*`threadpoolworker/starttpw`*, the JMX batch connector* `service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector`*, WLST, the batch run-status tables) are from the OUAF Server Administration Guide — verify exact names/flags for your version. MOS-gated go-live checklists, patches, and known-issue notes are pointed to, not reproduced.*

Step 1 — Build the go-live readiness checklist (`C2M Runbook - Readiness - Go-Live Checklist`)

1. Source the environment and run `OC2MHYPER-Discovery.sh` to capture current state: WebLogic servers

and the batch daemon, the JMX batch connector, configured integrations, backup presence, and the monitoring signals — read-only. This is the raw material for the checklist.

1. Assemble the checklist with an accountable owner and a go/no-go status per line, grouped by domain:

• **Environment** — WebLogic Admin + managed servers RUNNING; `threadpoolworker`/batch daemon

started on the designated node(s); certified stack confirmed.

• **Data & migration** — final data load/sync signed off; reconciliation counts agreed.

• **Integrations** — each inbound/outbound endpoint reachable and enabled; queues drained.

• **Batch estate** — the nightly schedule and dependencies in place; a smoke batch completes.

• **Security/SSO** — SSO, SSL/keystore, and access confirmed for production URLs.

• **Monitoring** — the health routine, alerting, and the monitoring platform (e.g. Dynatrace) live.

• **Backup & rollback** — backups taken and tested; the rollback documented.

• **Support readiness** — rota staffed, escalation matrix and severity model agreed, known-issue

register seeded.

1. Run `OC2MHYPER-Validation.sh` — it checks the technical readiness signals that back the checklist

and reports PASS/FAIL per signal. Do not proceed past a red technical item without a signed exception.

> ***Gate — Readiness.*** *Cutover starts only when every checklist item is green or carries a signed-off exception, and* `OC2MHYPER-Validation.sh` *returns overall PASS.*

Step 2 — Author the cutover plan (`C2M Runbook - Cutover - Plan`)

1. Write the cutover as a timeboxed, numbered sequence with a start time, a **go/no-go** decision

point partway through, and a hard **rollback deadline**. A typical sequence: freeze change → take backups → final data load/sync → promote configuration → enable integrations → start the batch daemon and `threadpoolworker` → run smoke checks → go/no-go → open to users.

1. Attach the **rollback**: the exact steps to restore the pre-cutover backups and revert config if

the go/no-go fails, with the person who authorizes rollback named.

1. Rehearse the whole sequence on the non-production dress rehearsal (Step 8) and time each step.

> ***Gate — Cutover.*** *Production is open only when the smoke checks pass at the go/no-go point; if not, execute the documented rollback rather than pressing on.*

Step 3 — Define the severity model (`C2M Runbook - Severity - Model`)

1. Fix four severities so severity is a rule, not an argument:

• **Sev-1** — production down or a critical business function (e.g. billing) blocked; immediate

page, incident bridge, continuous comms.

• **Sev-2** — major function degraded or at risk, workaround limited; fast page, frequent comms.

• **Sev-3** — a contained issue with a workaround; handled in-hours by the on-call owner.

• **Sev-4** — minor/cosmetic or a question; tracked, no page.

1. For each severity set the **response target** (time to acknowledge/engage) and the **comms

cadence**. These targets drive the escalation matrix.

Step 4 — Build the on-call rota and escalation matrix

1. **On-call rota** (`C2M Runbook - OnCall - Rota`): define primary and secondary for each support

window, by layer — application, middleware/WebLogic, database, integration, infrastructure — with current contacts and a **shift-handover** step (outgoing shift briefs incoming on open incidents and the day's health snapshot). Render the current window with `OC2MHYPER-ScopeHelper.sh --scope rota`.

1. **Escalation matrix** (`C2M Runbook - Escalation - Matrix`): for each severity, list who is engaged

and in what order and within what time — on-call owner → layer lead → incident commander → vendor/Oracle Service Request (MOS) for a suspected product defect. Make the MOS SR path explicit for defects; gated notes are pointed to, not reproduced.

Step 5 — Adopt the incident procedure (`C2M Runbook - Incident - Procedure`)

1. Standardize the incident lifecycle: **detect** (health routine, alert, or user report) → **triage

and set severity **(per the severity model)** → assign an owner **(per the rota)** → diagnose **(health scripts, logs, JMX, schema)** → mitigate/resolve **(workaround now, root-cause after)** → communicate **(per the severity's cadence)** → record** (timeline, cause, fix, and — if recurring — a known-issue entry).

1. For a major incident (Sev-1/Sev-2), open an **incident bridge**, name an **incident commander**

(coordinates, does not debug), and keep a running timeline. After resolution, run a brief **post-incident review** and feed any recurring pattern into the known-issue register.

Step 6 — Handle known issues (`C2M Runbook - KnownIssue - Register`)

1. Seed the register before go-live from testing/UAT defects and the version's known-issue MOS notes

— each entry: symptom, affected area, severity, **workaround**, and the tracking reference (a MOS note/patch number or an internal ticket). Do not reproduce gated content; point to it.

1. During hypercare, when an incident matches a known issue, apply the recorded workaround

immediately and log the recurrence; when a new recurring issue appears, add it. Render the current register and open incidents with `OC2MHYPER-Report.sh` (read-only).

Step 7 — Set up the day-2 build lane (`C2M Runbook - Day2 - Build Lane`)

1. Keep all further configuration and development in the **lower environments** (Dev/Test/UAT) during

hypercare; nothing is built directly in the stabilizing production system.

1. Promote day-2 changes to production **only through change control** and the existing

code-promotion path — same discipline as any production change: proven in a lower environment, backed up, with a rollback. List the day-2 environments and controls with `OC2MHYPER-ScopeHelper.sh --scope day2`.

1. Coordinate day-2 promotions with the on-call/hypercare team so a build change never collides with

incident work or the daily cutover of the health snapshot.

Step 8 — Configure the daily health/monitoring routine (`C2M Runbook - Health - Daily Routine`)

1. Fix the morning routine as a repeatable, read-only set — run it the same way every day:

• **WebLogic** — Admin and managed servers RUNNING (WLST); no failed deployments.

• **Batch** — last night's cycle completed; no errored threads; the Batch Job Submission backlog

drained (via the JMX batch connector and the run-status report).

• **Integration** — inbound/outbound queues within threshold; no stuck messages.

• **Database & disk** — schema reachable, key tablespaces and log/filesystem space within

threshold, no runaway sessions.

• **Errors** — the delta of new ERROR/FATAL lines in the OUAF/WebLogic logs since yesterday.

1. Produce a **green/amber/red health snapshot** each morning: `OC2MHYPER-Report.sh` renders the

operational summary and `OC2MHYPER-Report.sql` renders the batch/integration/database detail (both read-only). Attach the snapshot to the shift handover.

1. Any amber/red item opens an incident at the appropriate severity (Step 5).

Step 9 — Pilot the whole model on a non-production dress rehearsal

1. On the non-production environment, **rehearse the cutover plan** end to end (Step 2), time each

step, and prove the rollback works.

1. **Run the daily health routine** for several days against non-production so the snapshot,

thresholds, and scripts are proven before go-live.

1. **Tabletop the support structure**: walk one Sev-1 and one Sev-3 through the severity model,

rota, escalation matrix, incident procedure, and known-issue register — confirm each reaches the right owner within its target and is recorded.

1. Run `OC2MHYPER-Validation.sh` to confirm the technical readiness and health signals are present.

> ***Gate — Pilot.*** *The dress rehearsal passes only when: the cutover ran within its timebox and the rollback was proven; the health routine ran clean for the agreed number of days; and the Sev-1 and Sev-3 tabletops reached the right owners within target and were recorded.*

Step 10 — Expand into the live cutover and hypercare

1. Execute the live cutover per `C2M Runbook - Cutover - Plan` under change control with backups taken

first; make the go/no-go call at the decision point; roll back rather than force a failed cutover.

1. Enter hypercare: staff the rota, run the daily health routine every morning, handle incidents

through the procedure, keep the known-issue register current, and run day-2 builds through the lower-environment lane.

1. Re-run `OC2MHYPER-Validation.sh` and the health reports each day of hypercare; keep the snapshot in

the shift handover so health stays visible day over day.

1. **Exit hypercare to steady-state** only when the exit gate is met: health green for the agreed

number of consecutive days, incident rate below threshold, no open Sev-1/Sev-2, and the day-2 backlog controlled. The runbook then becomes the standing operations runbook.

> ***Gate — Steady-state exit.*** *Hypercare ends only when the exit criteria above are met; until then the elevated support model stays in force.*

Order of operations

```
Readiness checklist -> Cutover plan -> Severity model -> On-call + escalation
-> Incident procedure -> Known-issue register -> Day-2 lane -> Daily health
routine -> Pilot (dress rehearsal) -> Expand (live cutover + hypercare) ->
Steady-state exit
```

5. Testing Guide — Oracle C2M Hypercare & Production Support Runbook

Run these on the non-production dress rehearsal (Runbook Step 9) and again during live hypercare. Each test confirms one part of the operating model works end to end: readiness gated, cutover reversible, on-call and escalation reaching the right owner, incidents handled and recorded, day-2 builds staying in the lower environments, and health measured daily. Source the environment first: `splenviron.sh -e your-env`.

Test 1 — Readiness is actually gated

Do: Run `OC2MHYPER-Discovery.sh` and `OC2MHYPER-Validation.sh` against the environment; review the `C2M Runbook - Readiness - Go-Live Checklist` with owners. Expected: Every technical readiness signal returns PASS (WebLogic servers RUNNING, batch daemon up, JMX reachable, integrations reachable, backups present); every checklist line is green or carries a signed-off exception. A red technical item blocks cutover — the gate holds.

Test 2 — Cutover is reversible

Do: On the dress rehearsal, execute `C2M Runbook - Cutover - Plan` end to end, then execute the rollback from a mid-cutover point. Expected: The cutover runs within its timebox; the go/no-go decision point is exercised; the rollback restores the pre-cutover state cleanly from the tested backups. A failed go/no-go leads to rollback, not to pressing on.

Test 3 — On-call and escalation reach the right owner

Do: Tabletop one Sev-1 and one Sev-3. Render the current rota with `OC2MHYPER-ScopeHelper.sh --scope rota`; walk each incident through `C2M Runbook - Severity - Model` and `C2M Runbook - Escalation - Matrix`. Expected: Each severity engages the correct owner in the correct order within its response target; the Sev-1 reaches the incident commander and the MOS Service Request path is known; the Sev-3 is handled by the on-call owner in-hours. Ownership is immediate, not debated.

Test 4 — An incident is handled and recorded

Do: Run a simulated incident through `C2M Runbook - Incident - Procedure` (detect → triage → assign → diagnose → mitigate → communicate → record); render the summary with `OC2MHYPER-Report.sh`. Expected: The incident gets a severity, an owner, a timeline, a

resolution/workaround, and a written record; a recurring pattern produces a `C2M Runbook - KnownIssue - Register` entry.

Test 5 — A known issue is recognized, not re-diagnosed

Do: Trigger (or simulate) a symptom already in the known-issue register; have the on-call owner consult `OC2MHYPER-Report.sh` / the register. Expected: The symptom is matched to its register entry and the recorded workaround is applied in minutes; the recurrence is logged. It is not diagnosed from scratch.

Test 6 — Day-2 builds stay in the lower environments

Do: Run a representative day-2 change through the `C2M Runbook - Day2 - Build Lane`; list the lane with `OC2MHYPER-ScopeHelper.sh --scope day2`. Expected: The change is built and tested in a lower environment and promoted to production only through change control — nothing is built directly in production during hypercare.

Test 7 — The daily health routine produces a clean snapshot

Do: Run the routine (`OC2MHYPER-Report.sh` for the operational summary, `OC2MHYPER-Report.sql` for batch/integration/database detail) as it would run each morning. Expected: A green/amber/red snapshot covering WebLogic, batch completion and backlog, integration queues, database/disk, and error-log deltas. Green = stable; any amber/red opens an incident at the right severity. The routine runs the same way every day.

Test 8 — Health is stable across the hypercare window

Do: Run the daily routine for the agreed number of consecutive days during hypercare and track the snapshots. Expected: Health trends green with no open Sev-1/Sev-2 and an incident rate below threshold — the steady-state exit gate. A downward trend in daily amber/red items shows stabilization.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-7 pass on the non-production dress rehearsal: readiness gated, cutover reversible, on-call/escalation reach the right owner, an incident is handled and recorded, a known issue is recognized, day-2 stays in lower environments, the daily health snapshot is clean
Hypercare exit pass	Tests 1-7 continue to pass in live hypercare and Test 8 holds — health green for the agreed consecutive days, no open Sev-1/Sev-2, incident rate below threshold, day-2 backlog controlled

6. Troubleshooting Guide — Oracle C2M Hypercare & Production Support Runbook

Use `OC2MHYPER-Discovery.sh` and `OC2MHYPER-Validation.sh` for readiness/config state, `OC2MHYPER-Report.sh` for the daily health snapshot and the open-incident/known-issue summary, `OC2MHYPER-Report.sql` for batch/integration/database detail, and `OC2MHYPER-Remediation.sh` (dry-run by default) for safe operational alignments. Source the environment first: `splenviron.sh -e your-env`. MOS-gated go-live checklists, patches, and known-issue notes are pointed to, not reproduced.

A. Readiness checklist stays red

Symptom	Cause	Remediation
Validation fails on WebLogic	A managed server or the Admin server not RUNNING	Confirm server state via WLST; start the server in the change window; re-run `OC2MHYPER-Validation.sh`
Validation fails on batch	`threadpoolworker`/batch daemon not started on the designated node	Start the worker/daemon (`threadpoolworker`/`starttpw` — verify for your version); confirm the JMX batch connector responds
Validation fails on integration	An inbound/outbound endpoint unreachable or disabled	Confirm the endpoint URL/queue with the integration team; enable it; re-check
Checklist item red, no owner	Ownership not assigned	Assign an accountable owner per line before go/no-go; a red item needs a signed exception or a fix

B. Cutover go/no-go fails

• The smoke checks do not pass at the go/no-go point. **Do not press on.** Execute the documented rollback from `C2M Runbook - Cutover - Plan` — restore the pre-cutover backups and revert config — with the named authorizer's approval. Then diagnose on the restored environment, fix, and reschedule the cutover; a forced cutover past a failed go/no-go is how a bad night becomes a bad week.

C. An incident has no clear owner

• An issue arrives and the bridge spends its first hour deciding who owns it. The rota was not staffed for the window or not published. Render the current window with `OC2MHYPER-ScopeHelper.sh --scope rota`; if a layer has no primary/secondary, fix the `C2M Runbook - OnCall - Rota` and publish it. Escalation order comes from `C2M Runbook - Escalation - Matrix` keyed to the severity — set the severity first (Step 3) so the matrix routes it.

D. The same problem keeps being re-diagnosed

• A recurring symptom is investigated from scratch each time. It is not in the known-issue register,

or the register is not consulted. Add it to `C2M Runbook - KnownIssue - Register` with its workaround and tracking reference (a MOS note/patch or internal ticket — point to gated notes, do not copy them); make consulting `OC2MHYPER-Report.sh` / the register the first triage step.

E. Day-2 change destabilized production

• A configuration or code change went straight into the stabilizing production system and caused an

incident. Day-2 work must run through `C2M Runbook - Day2 - Build Lane`: built and tested in a lower environment, promoted only through change control with a rollback. Roll the change back, rebuild it in a lower environment, and re-promote through the lane; list the lane with `OC2MHYPER-ScopeHelper.sh --scope day2`.

F. Daily health shows amber/red

Symptom	Cause	Remediation
Batch amber/red	Last night's cycle did not complete, or backlog did not drain	Inspect run status and backlog with `OC2MHYPER-Report.sql`; check live threads via the JMX batch connector; open an incident at the right severity
Integration amber/red	A queue over threshold or a stuck message	Confirm the queue depth with the integration team; clear/replay per their runbook; track as an incident
Database/disk amber/red	Tablespace or log/filesystem space near threshold	`OC2MHYPER-Report.sql` shows space; engage the DBA; `OC2MHYPER-Remediation.sh` (dry-run) prints safe log/space cleanup targets — apply only in a change window
Error-log delta spikes	New ERROR/FATAL lines since yesterday	Review the delta; correlate with any incident; add a recurring signature to the known-issue register

G. JMX batch connector cannot connect

• `OC2MHYPER-Discovery.sh`/`OC2MHYPER-Validation.sh` cannot reach the batch JMX interface. Confirm the

RMI/JMX port is populated on the worker and the connector URL is right — `service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector` (verify the exact form for your version) — and that the port is reachable. Restart the worker in a change window if the port was not set; then the daily batch health check can read live threads.

H. Health will not stabilize / hypercare will not exit

• Health keeps producing amber/red or Sev-1/Sev-2 incidents recur, so the steady-state exit gate is

never met. Do not exit on the calendar. Work the top recurring signatures from the known-issue register to root cause (day-2 lane fixes promoted under change control), re-measure with the daily routine, and exit only when health is green for the agreed consecutive days with no open Sev-1/Sev-2 and a controlled day-2 backlog.

I. Diagnostics quick reference

```
splenviron.sh -e your-env                          # source the environment first
./OC2MHYPER-Discovery.sh                            # readiness/health current state (read-only)
./OC2MHYPER-Validation.sh                           # go-live readiness signals PASS/FAIL (read-only)
./OC2MHYPER-Report.sh                               # daily health snapshot + open incidents/known issues (
sqlplus user@db @OC2MHYPER-Report.sql               # batch/integration/database health detail (read-only)
./OC2MHYPER-ScopeHelper.sh --scope rota             # current on-call window contacts
./OC2MHYPER-ScopeHelper.sh --scope day2             # day-2 build lane environments/controls
./OC2MHYPER-Remediation.sh                          # dry-run by default; add the apply flag in a change wi
```

J. When to escalate

If readiness is green, the rota and escalation matrix are staffed, incidents are being handled through the procedure, and a symptom still recurs or health will not stabilize after remediation and day-2 fixes, capture `OC2MHYPER-Validation.sh`, `OC2MHYPER-Report.sh`, and `OC2MHYPER-Report.sql` output and review with the DBA, middleware, and integration teams. For a suspected product defect, use the My Oracle Support Service Request path in the escalation matrix and reference the version's known-issue notes — gated content is pointed to, not reproduced here.

7. Best Practices — Oracle C2M Hypercare & Production Support Runbook

Field-proven recommendations for a smooth stabilization from cutover through steady-state.

• Gate cutover; never start on hope. `C2M Runbook - Readiness - Go-Live Checklist` with an owner per

line and a signed go/no-go is the difference between a controlled cutover and an unpatched node found at 2 a.m. A pre-approved exception is fine; an unexamined red item is not.

• Make cutover reversible before you make it live. A timeboxed plan with an explicit go/no-go and a

tested rollback means a failed cutover is a rollback, not a crisis. A rollback that has not been walked on non-production is not a rollback.

• Rehearse the whole thing. The non-production dress rehearsal — cutover timed, health routine run for

days, Sev-1/Sev-3 tabletops — finds the gaps while they are cheap. Programs that skip the rehearsal discover the same gaps live.

• Make severity a rule, not an argument. `C2M Runbook - Severity - Model` decides response time, who

is paged, and comms cadence up front, so the incident bridge spends its first minutes fixing the problem, not classifying it.

• Give every incident an owner immediately. A staffed `C2M Runbook - OnCall - Rota` by layer, with a

primary and a secondary and a real shift handover, means no incident spends its first hour deciding who owns it. Brief the incoming shift on open incidents and the day's health snapshot.

• Separate command from debugging in a major incident. Name an incident commander who coordinates and

communicates while the engineers debug — one person doing both does neither well.

• Handle each incident once and write it down. Follow `C2M Runbook - Incident - Procedure` through to

record; a recurring pattern becomes a `C2M Runbook - KnownIssue - Register` entry so it is worked around next time, not re-diagnosed.

• Seed and keep the known-issue register. Load it before go-live from UAT defects and the version's

known-issue MOS notes (point to gated notes, do not copy them); each entry carries a workaround and a tracking reference. A recognized symptom is a two-minute fix; an unrecognized one is a fresh investigation.

• Keep day-2 work in the lower environments. During hypercare, build and test in Dev/Test/UAT and

promote to production only through change control. Building in a stabilizing production system trades a little speed now for instability you will pay for later.

• Measure health every morning, the same way. `C2M Runbook - Health - Daily Routine` — WebLogic, batch

completion and backlog, integration queues, database/disk, error-log deltas — turns "how are we doing?" into a green/amber/red snapshot. Attach it to the shift handover; any amber/red opens an incident.

• Prefer read-only diagnosis first. The discovery, validation, and report scripts read; remediation is

dry-run by default. Understand the state before you change it, and change production only under change control with a rollback ready.

• Exit hypercare on evidence, not on the calendar. Leave the elevated model only when health is green

for the agreed consecutive days, there are no open Sev-1/Sev-2, the incident rate is below threshold, and the day-2 backlog is controlled — then the runbook becomes the standing operations runbook.

• Escalate product defects the right way. When diagnosis points to a product bug, use the

My Oracle Support Service Request path in the escalation matrix and reference the known-issue register — do not sit on a defect or reproduce gated content locally.

8. Deliverables Summary — Oracle C2M Hypercare & Production Support Runbook

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The hypercare operating model and production-support structure (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites, published once
3. Runbook	The runbook itself: go-live readiness checklist, on-call/escalation model, incident + known-issue handling, day-2 build support in lower environments, and the daily health/monitoring routine — with gates
5. Testing Guide	Validation procedures with pass/fail gates (readiness gated, incidents handled, health stable)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for a smooth stabilization

Named objects

Object	Name
Go-live readiness gate	`C2M Runbook - Readiness - Go-Live Checklist`
Cutover sequence	`C2M Runbook - Cutover - Plan`
On-call schedule	`C2M Runbook - OnCall - Rota`
Escalation matrix	`C2M Runbook - Escalation - Matrix`
Severity model	`C2M Runbook - Severity - Model`
Incident lifecycle	`C2M Runbook - Incident - Procedure`
Known-issue register	`C2M Runbook - KnownIssue - Register`
Day-2 build lane	`C2M Runbook - Day2 - Build Lane`
Daily health snapshot	`C2M Runbook - Health - Daily Routine`

Automation scripts (WLST/Jython, shell & SQL — not PowerShell)

Script	What it does
`OC2MHYPER-Discovery.sh`	Captures go-live/health current state — WebLogic servers, batch daemon, JMX batch connector, integrations, backup presence, monitoring signals (read-only)

Script	What it does
`OC2MHYPER-Validation.sh`	Validates the technical go-live readiness signals behind the checklist and reports PASS/FAIL per signal (read-only)
`OC2MHYPER-Report.sh`	Renders the daily health snapshot and the open-incident / known-issue summary from the runbook files and live signals (read-only)
`OC2MHYPER-Report.sql`	Renders batch run status/backlog, integration queue depth, and database/space health from the C2M schema (read-only)
`OC2MHYPER-Remediation.sh`	Aligns safe operational settings (restart guidance, log/space cleanup targets, health-threshold config) to the standard; dry-run by default, applies only with an explicit flag in a change window
`OC2MHYPER-ScopeHelper.sh`	Renders the on-call rota window (`--scope rota`) and the day-2 build lane (`--scope day2`) from the scope file for pilot and hypercare

(Five automation scripts as specified — Discovery, Validation, Report, Remediation, ScopeHelper — with the Report delivered in two authentic forms, a shell operational summary and a SQL data report, per the C2M/WLST/SQL toolchain.)

What the buyer receives

- This runbook as a branded Word document and PDF.
- All automation scripts (shell / WLST-Jython / SQL) in a single ZIP.
- A repeatable operating model following: Assess, then Configure, then Pilot, then Validate, then Expand.

Outcome

A C2M go-live and hypercare the team can actually run — readiness proven and gated, cutover reversible, on-call and escalation structured, incidents handled and recorded, known issues worked around, day-2 builds kept safely in the lower environments, and health measured green every morning — from cutover through stabilization to a clean steady-state exit.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MHYPER-Discovery.sh

Purpose: Capture the current state that backs the go-live readiness checklist and the daily health routine for an Oracle Utilities C2M environment: WebLogic server state, the batch daemon / threadpoolworker, the JMX batch connector, configured integrations, backup presence, and the monitoring signals. Read-only — it changes nothing. Run it before building the readiness checklist (Runbook Step 1) and each hypercare morning. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Discovery.sh Optional: DB_CONN=user/pass@your-db to include batch/integration status from the schema. Optional: JMX_URL=service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector EXPECTED RESULTS Prints: environment sourced OK and OUAF/WebLogic utilities on PATH; the batch daemon / threadpoolworker presence; the JMX batch connector reachability (if JMX_URL set); the configured log locations and recent backup presence; and (if DB_CONN set) recent batch run status. PASS = environment sourced and signals collected. FAIL = environment not sourced or utilities not found — fix before continuing. NOTES White-label, repeatable discovery — the same current-state capture every environment and every hypercare morning. Read-only where it reads live systems: it only SELECTs from the schema, queries JMX with a summary action, and reads config/log paths. Utility and connector names are from the OUAF Server Administration Guide — verify for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Discovery.sh Optional: DB_CONN=user/pass@your-db to include batch/integration status from the schema. Optional: JMX_URL=service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector EXPECTED RESULTS Prints: environment sourced OK and OUAF/WebLogic utilities on PATH; the batch daemon / threadpoolworker presence; the JMX batch connector reachability (if JMX_URL set); the configured log locations and recent backup presence; and (if DB_CONN set) recent batch run status. PASS = environment sourced and signals collected. FAIL = environment not sourced or utilities not found — fix before continuing. NOTES White-label, repeatable discovery — the same current-state capture every environment and every hypercare morning. Read-only where it reads live systems: it only SELECTs from the schema, queries JMX with a summary action, and reads config/log paths. Utility and connector names are from the OUAF Server Administration Guide — verify for your version.

OC2MHYPER-Remediation.sh

Purpose: Print (and, only when explicitly asked, apply) safe operational alignments that hypercare commonly needs: the batch-worker restart guidance, log/space cleanup targets when the daily health snapshot is amber/red on space, and the health-threshold config used by the report scripts. DRY-RUN by default: it prints the exact change it WOULD make and changes nothing. It never touches business logic and never restarts a running server itself. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Remediation.sh # DRY-RUN: prints intended actions only ./OC2MHYPER-Remediation.sh --apply # applies safe file writes (change window only) Override thresholds: TS_PCT=85 LOG_RETAIN_DAYS=14 ./OC2MHYPER-Remediation.sh EXPECTED RESULTS DRY-RUN prints: the standard health thresholds, the log/space cleanup targets (old log files beyond retention), and the exact worker restart command the admin must run — and exits without

changing anything. With --apply it writes the health-threshold config file (backing up any existing one) and prints the cleanup and restart steps for the admin to perform under change control. It never deletes files or restarts services itself. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It aligns only operational settings and prints manual steps for anything impacting a running service; it never changes business data or logic. Utility names are from the OUAF Server Administration Guide — verify for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Remediation.sh # DRY-RUN: prints intended actions only ./OC2MHYPER-Remediation.sh --apply # applies safe file writes (change window only) Override thresholds: TS_PCT=85 LOG_RETAIN_DAYS=14 ./OC2MHYPER-Remediation.sh EXPECTED RESULTS DRY-RUN prints: the standard health thresholds, the log/space cleanup targets (old log files beyond retention), and the exact worker restart command the admin must run — and exits without changing anything. With --apply it writes the health-threshold config file (backing up any existing one) and prints the cleanup and restart steps for the admin to perform under change control. It never deletes files or restarts services itself. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It aligns only operational settings and prints manual steps for anything impacting a running service; it never changes business data or logic. Utility names are from the OUAF Server Administration Guide — verify for your version.

OC2MHYPER-Report.sh

Purpose: Render the operational side of the daily health routine (C2M Runbook - Health - Daily Routine): WebLogic server presence, the batch worker and its live JMX summary, error/ fatal log deltas since yesterday, and the current open-incident and known-issue summary read from the runbook files. Read-only — it reads live signals and text files, changing nothing. Run it each hypercare morning; pair it with OC2MHYPER-Report.sql for DB detail. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Report.sh Optional: JMX_URL=service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector Optional: RUNBOOK_DIR=./runbook (dir holding incident/known-issue registers) Optional: LOG_DIR=path (OUAF/WebLogic log dir; defaults to your-splebase/logs) EXPECTED RESULTS Prints a green/amber/red-style snapshot: batch worker running (or not), JMX live-thread summary, count of new ERROR/FATAL log lines since yesterday, and the open-incident / known-issue counts from the registers. GREEN = worker up, JMX clean, no new errors, no open Sev-1/Sev-2. Any amber/red line is a candidate incident. Attach the snapshot to the shift handover. NOTES White-label, repeatable health snapshot — the same morning routine every day of hypercare. Read-only where it reads live systems (JMX summary, log scan) and text registers. Utility and connector names are from the OUAF Server Administration Guide — verify for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Report.sh Optional: JMX_URL=service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector Optional: RUNBOOK_DIR=./runbook (dir holding incident/known-issue registers) Optional: LOG_DIR=path (OUAF/WebLogic log dir; defaults to your-splebase/logs) EXPECTED RESULTS Prints a green/amber/red-style snapshot: batch worker running (or not), JMX live-thread summary, count of new ERROR/FATAL log lines since yesterday, and the open-incident / known-issue counts from the registers. GREEN = worker up, JMX clean, no new errors, no open Sev-1/Sev-2. Any amber/red line is a candidate incident. Attach the snapshot to the shift handover. NOTES White-label, repeatable health snapshot — the same morning routine every day of hypercare. Read-only where it reads live systems (JMX summary, log

scan) and text registers. Utility and connector names are from the OUAF Server Administration Guide — verify for your version.

OC2MHYPER-Report.sql

Purpose: Render the database-side detail for the daily health routine (C2M Runbook - Health - Daily Routine): batch run status and backlog for last night's cycle, integration queue depth, and database/space health signals. Read-only — SELECT only, it changes no data. Run it each hypercare morning alongside OC2MHYPER-Report.sh to produce the green/amber/red health snapshot. HOW TO RUN Via SQL*Plus or SQLcl against the C2M schema (a read-only account is fine): sqlplus user/pass@your-db @OC2MHYPER-Report.sql (or sql user/pass@your-db @OC2MHYPER-Report.sql with SQLcl) EXPECTED RESULTS Four sections print: (1) batch runs in the last 24h with status and duration, (2) failed/error threads, (3) Batch Job Submission backlog, (4) tablespace space usage. GREEN = all controls Complete, backlog = 0, no space over threshold. AMBER/RED = any error thread, a non-zero backlog, or a tablespace near full — each opens an incident at the appropriate severity. NOTES White-label, repeatable health report — the standing morning query set, the same every day. Read-only against live systems. Table/column names follow the OUAF/C2M batch model and Oracle data-dictionary views — verify object names for your version (they are cited, not invented). Integration/queue tables vary by adapter; the queue section is a template to point at your integration staging table.

How to run: Via SQL*Plus or SQLcl against the C2M schema (a read-only account is fine): sqlplus user/pass@your-db @OC2MHYPER-Report.sql (or sql user/pass@your-db @OC2MHYPER-Report.sql with SQLcl) EXPECTED RESULTS Four sections print: (1) batch runs in the last 24h with status and duration, (2) failed/error threads, (3) Batch Job Submission backlog, (4) tablespace space usage. GREEN = all controls Complete, backlog = 0, no space over threshold. AMBER/RED = any error thread, a non-zero backlog, or a tablespace near full — each opens an incident at the appropriate severity. NOTES White-label, repeatable health report — the standing morning query set, the same every day. Read-only against live systems. Table/column names follow the OUAF/C2M batch model and Oracle data-dictionary views — verify object names for your version (they are cited, not invented). Integration/queue tables vary by adapter; the queue section is a template to point at your integration staging table.

OC2MHYPER-ScopeHelper.sh

Purpose: Produce the bounded, reusable scope hypercare runs on: the current on-call rota window (who is primary/secondary by layer) and the day-2 build lane (which lower environments and controls further work flows through during hypercare). It reads one scope file and emits a clean, de-duplicated list, so the rota and the day-2 lane are explicit and auditable rather than remembered. Read-only — it only reads the scope file. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-ScopeHelper.sh --scope rota # current on-call window contacts by layer ./OC2MHYPER-ScopeHelper.sh --scope day2 # day-2 build lane environments/controls Scope file (default ./oc2mhyper-scope.conf) lines: KIND,LAYER_OR_ENV,NAME,CONTACT_OR_CONTROL rota lines: rota,middleware,Primary Name,pager-or-email day2 lines: day2,UAT,config-change-name,change-ticket EXPECTED RESULTS For --scope rota: the current window's primary/secondary per layer (application, middleware, database, integration, infrastructure) with contacts. For --scope day2: the lower environments and the day-2 controls flowing through change control. If the scope file is missing it prints a template to create. Read-only — no live-system changes. NOTES White-label, repeatable scoping — the same file drives the rota window and the day-2 lane through hypercare, so both are explicit and auditable. Read-only. No live-system changes. Names are yours to fill from the runbook (rota, escalation matrix, day-2 lane).

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-ScopeHelper.sh --scope rota # current on-call window contacts by layer ./OC2MHYPER-ScopeHelper.sh --scope day2 # day-2 build lane environments/controls Scope file (default ./oc2mhyper-scope.conf) lines: KIND,LAYER_OR_ENV,NAME,CONTACT_OR_CONTROL rota lines: rota,middleware,Primary Name,pager-or-email day2 lines: day2,UAT,config-change-name,change-ticket EXPECTED RESULTS For --scope rota: the current window's primary/secondary per layer (application, middleware, database, integration, infrastructure) with contacts. For --scope day2: the lower environments and the day-2 controls flowing through change control. If the scope file is missing it prints a template to create. Read-only — no live-system changes. NOTES White-label, repeatable scoping — the same file drives the rota window and the day-2 lane through hypercare, so both are explicit and auditable. Read-only. No live-system changes. Names are yours to fill from the runbook (rota, escalation matrix, day-2 lane).

OC2MHYPER-Validation.sh

Purpose: Validate the technical signals behind the go-live readiness checklist and report PASS/FAIL per signal: WebLogic servers reachable, the batch daemon / threadpoolworker running, the JMX batch connector reachable, the integration endpoints reachable, and the runbook files present. Read-only — it validates state and changes nothing. Run it at the readiness gate (Runbook Step 1) and each day of hypercare. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Validation.sh Optional: JMX_URL=service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector Optional: ADMIN_URL=t3://your-host:your-admin-port (WebLogic Admin server, WLST reachability) Optional: INTEGRATION_URLS="https://your-endpoint-1 https://your-endpoint-2" (reachability) Optional: RUNBOOK_DIR=./runbook (dir holding the runbook files to confirm present) EXPECTED RESULTS For each check prints PASS or FAIL: WebLogic reachable, batch worker running, JMX reachable, integration endpoints reachable, runbook files present and non-empty. Overall PASS = every readiness signal green — cutover may proceed. Any FAIL points to the checklist item to close (or a signed exception) before go/no-go. NOTES White-label, repeatable validation — the same readiness checks every environment, every day. Read-only where it reads live systems (WLST connect test, JMX summary, endpoint reachability). Utility/connector names are from the OUAF Server Administration Guide — verify for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MHYPER-Validation.sh Optional: JMX_URL=service:jmx:rmi:///jndi/rmi://your-host:your-jmx-port/oracle/ouaf/batchConnector Optional: ADMIN_URL=t3://your-host:your-admin-port (WebLogic Admin server, WLST reachability) Optional: INTEGRATION_URLS="https://your-endpoint-1 https://your-endpoint-2" (reachability) Optional: RUNBOOK_DIR=./runbook (dir holding the runbook files to confirm present) EXPECTED RESULTS For each check prints PASS or FAIL: WebLogic reachable, batch worker running, JMX reachable, integration endpoints reachable, runbook files present and non-empty. Overall PASS = every readiness signal green — cutover may proceed. Any FAIL points to the checklist item to close (or a signed exception) before go/no-go. NOTES White-label, repeatable validation — the same readiness checks every environment, every day. Read-only where it reads live systems (WLST connect test, JMX summary, endpoint reachability). Utility/connector names are from the OUAF Server Administration Guide — verify for your version.